Technical report — AI Fridge Chef

A deeper write-up than the README: what I built, how it works, and what I learned about Apertus along the way.

- **Track:** Track 2B — Own Project
- **Event:** Online
- **Team:** AI Fridge Chef — William Cheung
- **Demo:** https://youtu.be/bMZAZZVtkWw
- **Live deployment:** https://aifridgechef-d19909-wwmcheung.apps.ai-3.kvant.cloud

1. Summary

AI Fridge Chef turns a photo of your fridge, freezer, kitchen cupboard, or pantry into meal recipes and cooking suggestions. The user uploads a photo (or speaks and types), and Apertus detects the food items and builds an updatable food inventory. The app then generates recipes tailored to that inventory — and, optionally, to the user's dietary restrictions, medical considerations, cuisine preference, and serving size. A conversational "Fridge Chef" chatbot assistant answers follow-up questions grounded in the same inventory. Generated recipes are available in English and the four Swiss national languages (German, French, Italian, Romansh). Every AI capability — vision, speech, recipe generation, chat, and a reasoning-based chat summary — runs on the Apertus model family through the Swiss CSCS inference endpoint, and the application is deployed on a sovereign Swiss cloud, keeping the entire stack within Swiss jurisdiction.

2. Architecture

The app is a single Gradio web application (Python) served on port 7860. It is stateless apart from per-session UI state; all AI work is delegated to the remote Apertus endpoint over an OpenAI-compatible API (via LangChain's `ChatOpenAI`). There is no local model, no database, and no training component.

Data flow:

1. **Input** — the user provides a fridge, etc. photo, a spoken phrase (microphone), or typed text.
2. **Inventory building** — Apertus vision detects food items from the image; voice is transcribed by Apertus and the text routed into a natural-language inventory update; the result is a structured JSON inventory the user can review and update.
3. **Recipe generation** — on request, the current inventory plus any selected dietary restrictions, medical diagnoses, cuisine, and serving size, together with any previously generated recipes, are

sent to Apertus, which returns three recipes as structured JSON. Results are rendered to HTML and can be downloaded as a recipe book.
4. **Chat** — a conversational assistant answers inventory-grounded questions and can produce recipes directly in the conversation which can be further refined by the user.
5. **Chat summary** — on demand, the conversation is summarized (Markdown → HTML) and offered as a download, using the Apertus reasoning model.

Key modules: `gradio_ui.py` (UI and event wiring), `llm.py` (model factory, retries, rate limiting), `vision.py` (image detection), `audio_transcription.py` and `audio_inventory_update.py` (voice), `food_inventory.py` (inventory parsing), `recipes_generator.py` (recipes), `chef_chat.py` (chat and summary), `translations.py` (localized UI/recipe labels). Prompts live in `prompts/*.prompt.txt`.

Target architecture

The app is a self-contained Docker container whose only runtime dependency is an Apertus inference endpoint reachable over the network. That makes it deployable in these architectures:

- **On-premise** — the container can run on an organisation's own infrastructure, under its own administration, pointed at a reachable Apertus endpoint (a hosted endpoint, or an Apertus deployment on the same infrastructure).
- **Sovereign Swiss cloud** — the architecture actually used for this submission. The application is deployed and runs on the sovereign Swiss cloud operated by Phoenix Technologies AG (Pfäffikon SZ, Switzerland), on an OpenShift cluster, with all Apertus inference running on the Swiss National Supercomputing Centre (CSCS) endpoint. Both the hosting and the inference therefore remain under Swiss jurisdiction with Swiss data residency.

External dependencies, separating build time from runtime:

- **Build time:** a Docker image is built from `src/` on a `python:3.12-slim-bookworm` base; Python dependencies are installed from `requirements.txt` (pinned versions). The `.env` configuration (including the Apertus API key and endpoint) is baked into the image at build time.
- **Runtime:** the only external dependency is outbound HTTPS to the Apertus CSCS inference endpoint (`APERTUS_BASE_URL`). The container runs the Gradio server and orchestrates API calls; no other network services are required.

Cost and scalability

Because all heavy inference runs remotely on the Apertus endpoint, the application itself is light: an idle container sits around 190 MB of RAM with near-zero CPU, and the image is roughly 1 GB. Load scales by running more identical containers — the app holds no server-side session state, so instances need no coordination and sit naturally behind a load balancer. Inference cost is actively managed by matching

each task to the smallest model that does it well (the per-task model split in Section 3), so spend tracks the value of each call rather than defaulting to the biggest model everywhere.

3. Use of Apertus

- **Models:**
  - `swiss-ai/Apertus-v1.5-70B` (instruct) — the main user-facing model: image food detection, voice transcription, recipe generation, and the chef chat.
  - `swiss-ai/Apertus-v1.5-8B` (lite) — lightweight structured parsing: turning a photo-detected food list into the inventory, and applying spoken inventory updates.
  - `swiss-ai/Apertus-v1.5-70B-thinking` (reasoning) — reserved for the single task that benefits from multi-step reasoning: the downloadable chat summary.
- **How it is used:** inference only (no fine-tuning or training). The app uses prompt engineering with principle-based instructions, structured JSON output parsing, and multimodal inputs (image and audio sent via the OpenAI-compatible `image_url` / `audio_url` message format).
- **Where it runs:** hosted endpoint — the CSCS inference service, OpenAI-compatible API, accessed through LangChain's `ChatOpenAI`.

Design rationale for the three-model split (the cost and capability strategy alluded to in Section 2): common, high-frequency parsing operations — turning a photo-detected food list into the inventory, applying a spoken inventory update — run on the cheap, fast 8B model, where speed and low cost matter more than nuance. The quality-sensitive, user-visible generation (image food detection, recipe generation, the chef chat) runs on the 70B instruct model, where output quality is worth the extra cost and latency. The reasoning model is reserved for the one task that genuinely needs multi-step reasoning — the chat summary, which must group refined recipe variations and condense a conversation — and is deliberately kept off the interactive paths because, through the CSCS gateway, long reasoning requests hit an upstream timeout (~50–60s); the chat summary tolerates the latency because it is a non-interactive, user-initiated download.

Multilingual use. The selected language is threaded through the recipe, chat, vision, and inventory-update prompts so Apertus produces output natively in the chosen language (English plus the four Swiss national languages) rather than translating after the fact. For voice, the transcription prompt instructs the model to preserve whatever language is spoken, so the language is detected from the audio with no explicit selection. The recipe and chat prompts also apply Swiss-locale conventions for the four national languages (e.g. Swiss-German `ss` instead of `ß`, Swiss meal names).

Robustness details: `temperature=0` for determinism; a client-side rate limiter plus exponential-backoff retries on transient errors; and a delimiter (`<|inner_suffix|>`) used to strip the reasoning model's thinking trace from its final answer. Prompts are plain text files under `prompts/`, so the exact wording needed to rebuild the setup is in the repository.

4. Data

The app does not use a training or evaluation dataset. It processes only user-provided, in-session inputs (a photo, a voice clip, typed text) and small static reference lists shipped in `data/` (cuisine types, dietary restrictions, medical diagnoses, serving sizes) that populate UI dropdowns.

The app does not ask for or infer any personal information, so there is no human-subjects data and no non-redistributable dataset to flag.

5. Evaluation of Apertus

Because the app is prompt-driven inference against a hosted model (not a trained system with a baseline), I evaluated Apertus **behaviourally**: I exercised each feature with many prompt revisions and recorded where the model's output was reliable and where it was not. The results below are an honest account of Apertus 1.5's strengths and the areas where it has headroom, as experienced through this application. (Testing context: CSCS endpoint, `temperature=0`, instruct and reasoning variants.)

What worked well:

- **Multilingual generation.** Recipes and chat replies are produced natively in the requested language across English and the four Swiss national languages, including low-resource Romansh.
- **Structured JSON output.** Recipe generation and inventory parsing return parseable JSON reliably.
- **Image food detection.** Produced stable, usable inventories from photos (after prompt hardening to reject non-food, see below).
- **Conversational grounding.** The chat assistant answers follow-up questions grounded in the user's inventory and retains constraints across turns.

Where Apertus has headroom — feedback for the Apertus team. Across development I recorded 16 reproducible model behaviours, spanning language/localization (a "Switzerland locale" instruction biasing *language* selection; wrong language after a mid-conversation switch; Swiss "ss" orthography not honoured), instruction-following (macro percentages not summing to 100, serving-size mappings ignored, invented JSON fields, near-duplicate outputs under an exclusion list), grounding and safety (ignoring low-sodium/medical constraints — including an unsafe potassium-chloride substitute suggestion — and building recipes on unidentified "leftover" contents), multimodal (unreliable multilingual speech recognition, e.g. a clearly spoken French "gâteau chocolat" decoded as "ketorolac"; and non-food images such as gold coins detected as food), and operability (reasoning-mode requests timing out at the CSCS gateway). Each is documented with reproduction steps, the model(s) affected, and concrete evidence in docs/apertus-findings.md.

These findings directly shaped the architecture: the model-per-task split, the transcription-first voice flow (so a mis-hearing never acts unseen), and keeping the locale instruction only where the language is an explicit input.

6. Limitations

The app's constraints stem from the underlying model and platform (detailed in Section 5), and the app is designed to handle them responsibly rather than hide them:

- **Generated recipes are suggestions, not professional advice.** Because an LLM's nutrition figures and dietary/medical reasoning cannot be guaranteed, the app deliberately keeps the user in control — it presents recipes with a clear medical disclaimer and an editable inventory, so the person always reviews and decides. This is the correct posture for a health-adjacent assistant.
- **Voice input is treated as a draft.** Given Apertus's current speech-recognition limits for short multilingual phrases, the app intentionally routes transcription into an editable text box the user confirms before anything changes, rather than acting on audio directly. Voice is a convenience layer, and the design prevents a mis-hearing from ever altering the inventory unseen.
- **The reasoning model is used only where its latency is acceptable.** The Apertus reasoning model is powerful but, through the CSCS gateway, too slow for interactive use; the app reserves it for the asynchronous chat-summary download, where a short wait is fine, and uses faster models everywhere else. This is a deliberate model-per-task choice, not a shortcoming.

7. Reproducibility

- **Run it:** from the `track_2b/` directory, `make run` builds the Docker image from `src/` and runs the container, exposing the UI at `http://localhost:7860`. A valid Apertus API key must be set in `src/.env` before building (it is baked into the image). Full instructions, including a Docker-only alternative, are in the README.
- **Environment:** Docker; `python:3.12-slim-bookworm` base; pinned dependencies in `requirements.txt`; Gradio UI on port 7860.
- **Determinism:** `temperature=0` for all calls. Note that hosted inference can still exhibit minor non-determinism, and several model behaviours documented above are themselves non-deterministic across identical requests.

8. Next steps

AI Fridge Chef is already a working, deployed, end-to-end product. With the backing of this hackathon — prize funding and continued Apertus/CSCS credits — I would grow it from a single-session cooking assistant into a full Swiss-first kitchen companion. The following features all build directly on the strengths the app already demonstrates (multimodal input, multilingual output, inventory grounding, and sovereign Swiss hosting):

- **Persistent, expiry-aware inventory and waste reduction.** Give each user a saved inventory that carries across sessions, with estimated expiry dates and gentle "use these first" nudges. This turns a one-shot recipe generator into a daily food-waste-reduction tool — a concrete sustainability outcome, and a natural fit for Apertus-driven suggestions grounded in what is actually about to spoil.
- **Shopping list and meal planning.** Let the user plan several meals for the week and automatically generate a consolidated shopping list for only the ingredients they are missing, in their language. This extends the existing recipe pipeline from "what can I cook now" to "what should I buy and cook this week."
- **Household and dietary profiles.** Save per-person dietary restrictions and medical considerations (allergies, low-sodium, diabetic, etc.) so recipes are tailored to a whole household at once. This deepens the medical/dietary personalization the app already supports and makes it genuinely useful for families and care settings.
- **Nutrition and dietary-compliance validation.** Add a verification layer that checks recipe nutrition and flags dietary/medical conflicts before results are shown, pairing Apertus's generation with deterministic safety checks — the step that moves the app from "assistant with a disclaimer" toward something dependable in health-sensitive contexts.
- **Deeper Swiss-regional and seasonal cuisine.** Build out curated Swiss regional and seasonal recipe knowledge (sourced responsibly, not model-invented) so the app showcases authentic Swiss cooking — reinforcing the sovereign-Swiss positioning and giving Apertus grounded regional context to work from.

Each of these is incremental on top of the current architecture: no rewrite, just more value layered onto a foundation that already works and is already live on sovereign Swiss infrastructure.

License

Creative Commons Attribution 4.0 (CC-BY-4.0). All HackApertus projects are open-sourced.

References

- Apertus v1.5 model card: https://huggingface.co/swiss-ai/Apertus-v1.5-70B
- Swiss National Supercomputing Centre (CSCS), which hosts the Apertus inference endpoint: https://www.cscs.ch/
- Phoenix Technologies AG — sovereign Swiss cloud: https://phoeniqs.com